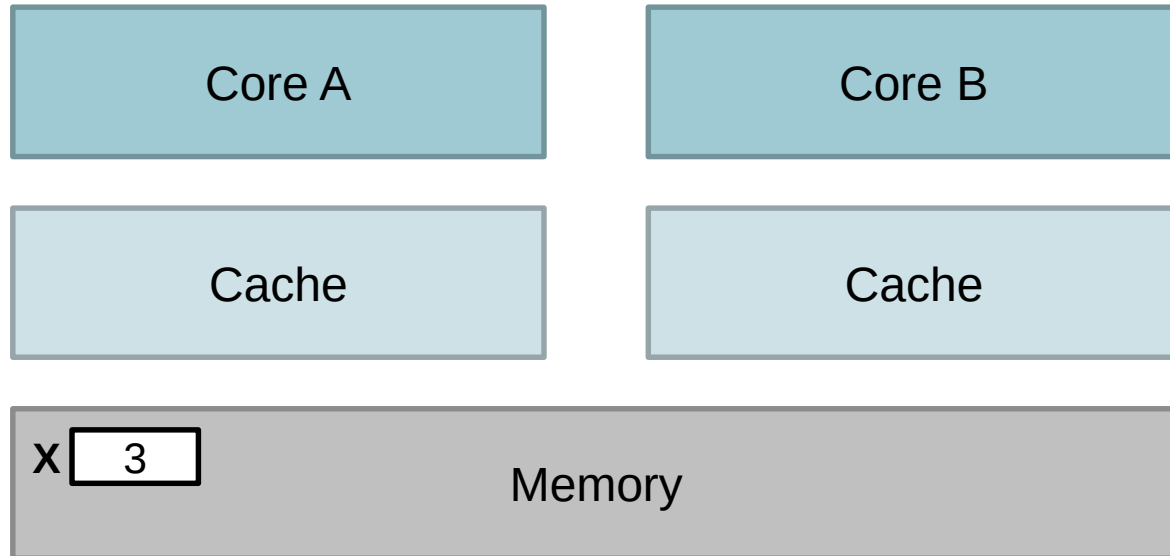


Lecture 34: Cache Coherence, Cache-aware Optimizations

CMPS 221 – Computer Organization and Design

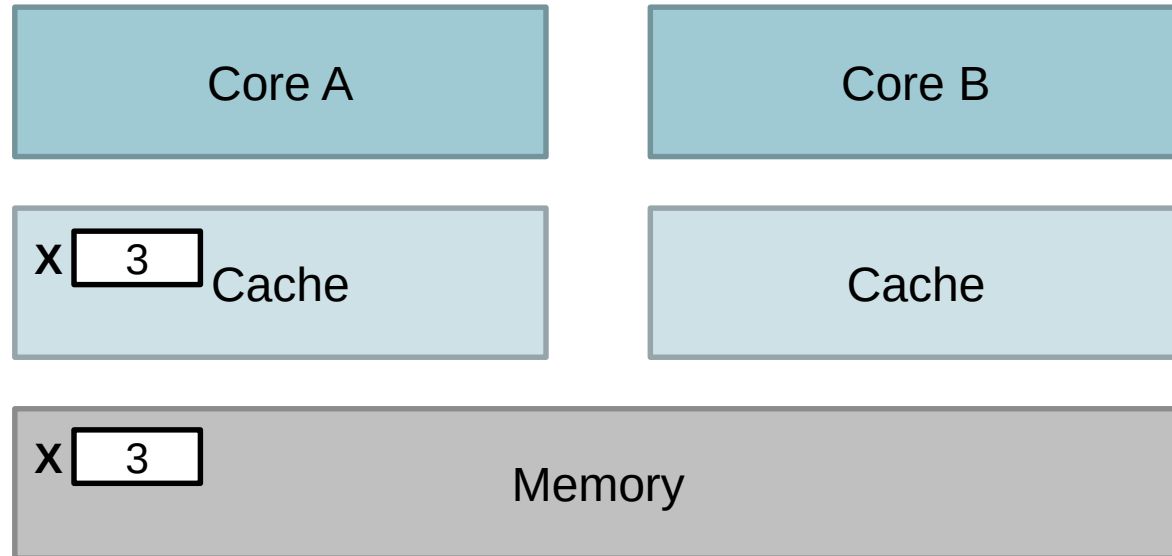
Cache Coherence Problem

- Consider two cores with private caches



Cache Coherence Problem

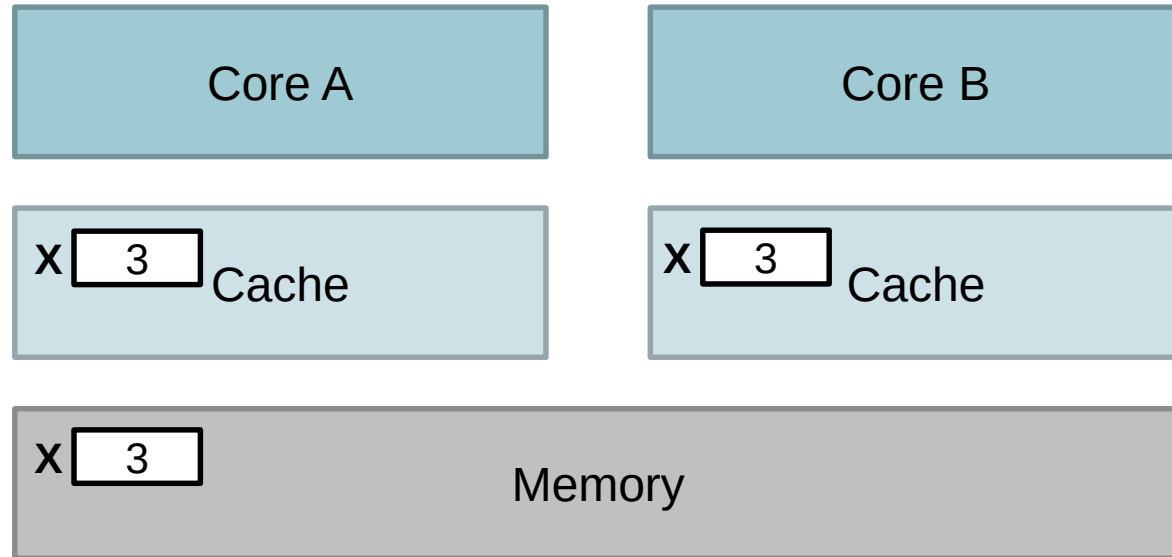
- Consider two cores with private caches



- Core A reads X

Cache Coherence Problem

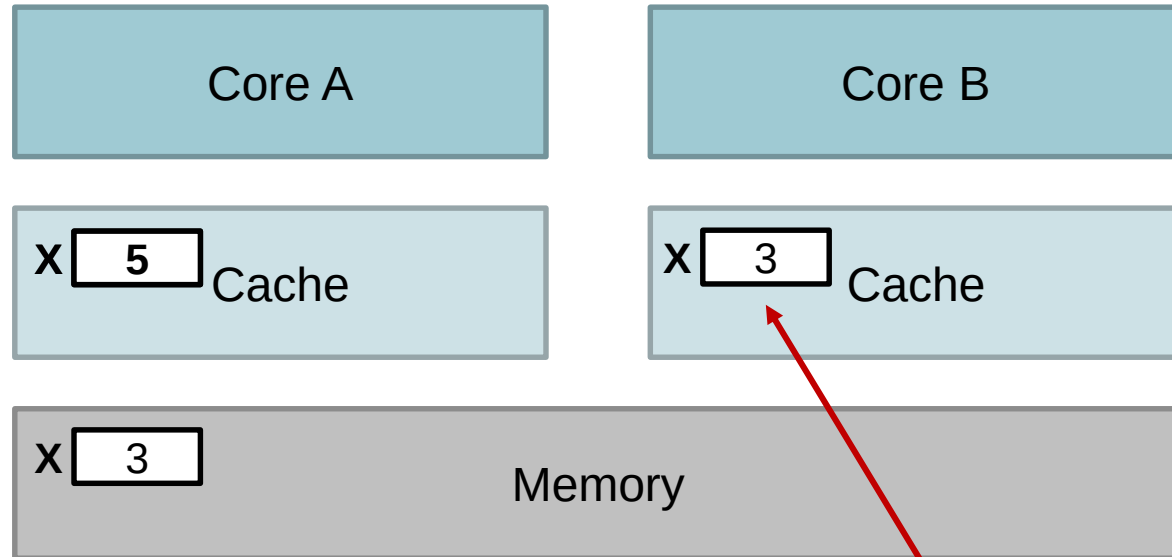
- Consider two cores with private caches



- Core A reads X
- Core B reads X

Cache Coherence Problem

- Consider two cores with private caches



- Core A reads X
- Core B reads X
- Core A writes to X

Value in Core B's cache becomes outdated. Core B should not be allowed to read outdated value.

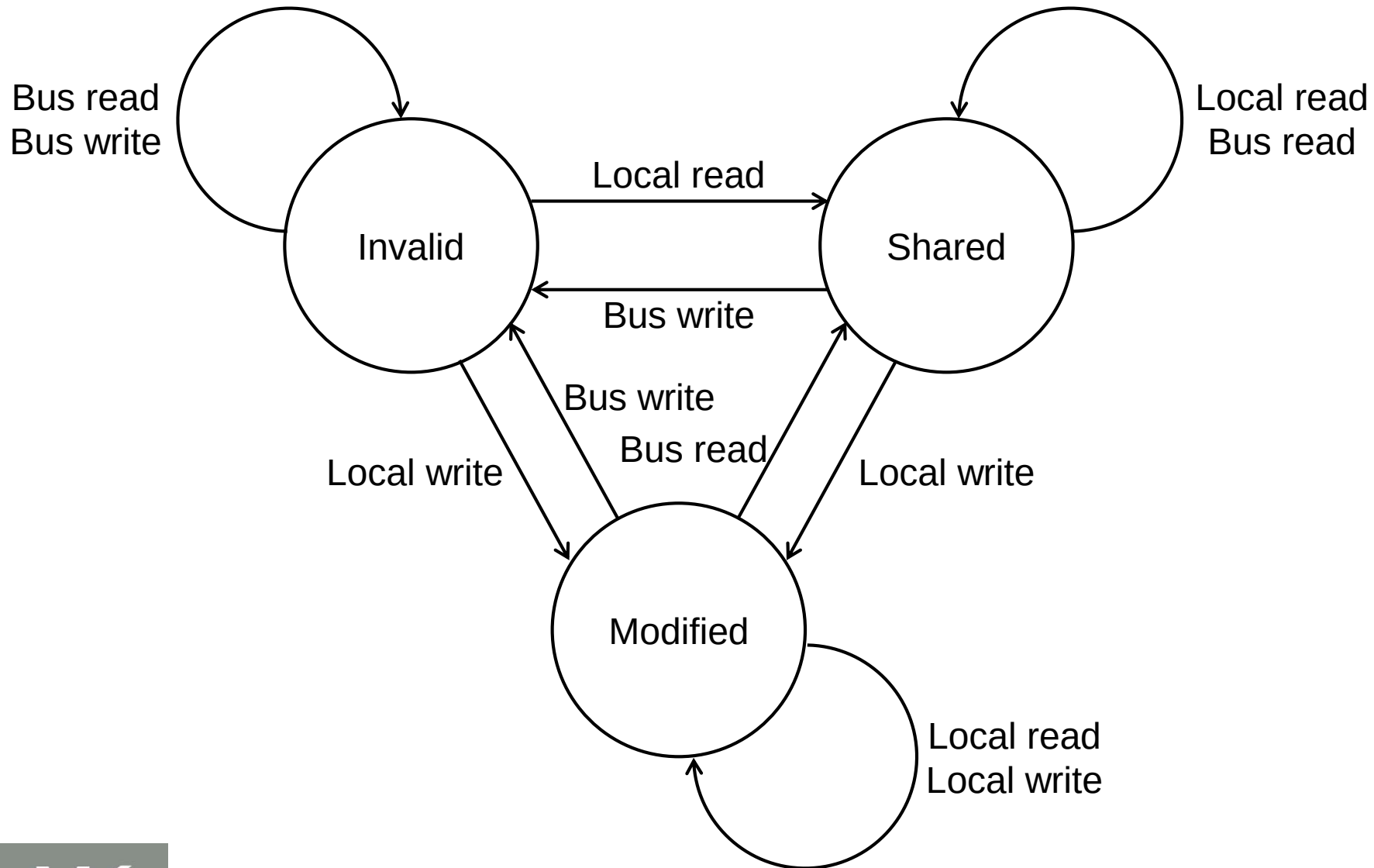
Cache Coherence Protocols

- A **coherence protocol** is a sequence of operations performed by caches in multiprocessors to ensure coherence
- Two major categories of protocols:
 - **Snooping protocols**
 - Each cache monitors reads/writes on a bus shared across all caches
 - **Directory-based protocols**
 - Caches and memory record sharing status of blocks in a centralized directory

Snooping Protocol Example: MSI

- A simple snooping protocol is **MSI**
 - Each block can be in one of three states:
 - **Modified**: The block is in the cache and has been modified
 - **Shared**: The block is in the cache but has not been modified
 - **Invalid**: The block is not in the cache
 - A block transitions between states based on the following events (with simplifying assumptions)
 - **Local read**: the block is read in the current cache
 - **Local write**: the block is written in the current cache
 - **Bus read**: the block is read in another cache
 - **Bus write**: the block is written in another cache

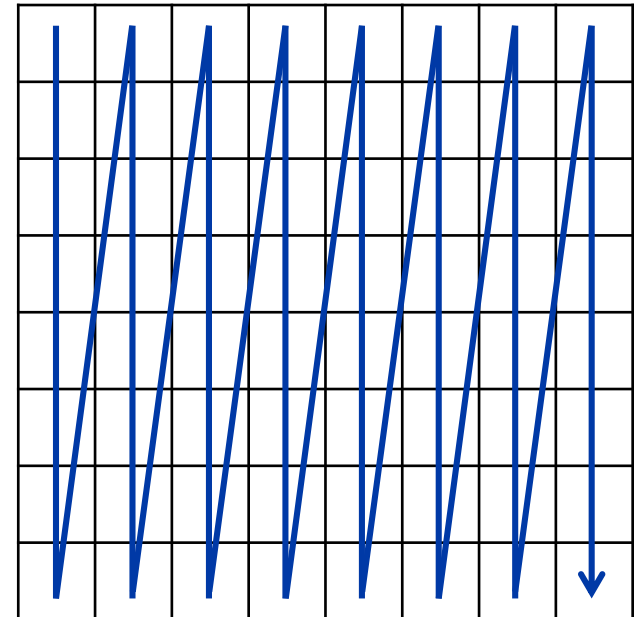
MSI State Diagram



Adding Elements of 2D Array

- Consider the following code that adds the elements of a 2D array

```
float sum = 0.0f;
for(unsigned int col = 0; col < 4096; ++col) {
    for(unsigned int row = 0; row < 4096; ++row) {
        sum += A[row][col];
    }
}
```

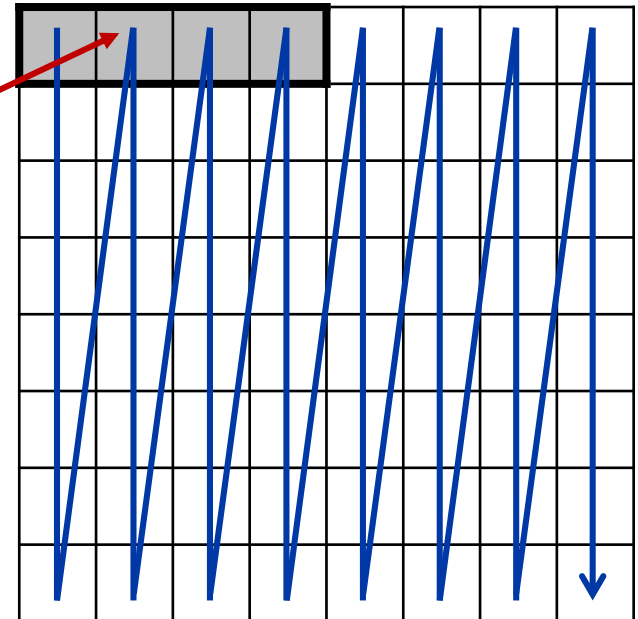


Adding Elements of 2D Array

- Consider the following code that adds the elements of a 2D array

```
float sum = 0.0f;  
for(unsigned int col = 0; col < 4096; ++col) {  
    for(unsigned int row = 0; row < 4096; ++row) {  
        sum += A[row][col];  
    }  
}
```

Data in a previously loaded cache line will be evicted by the time we come back for it

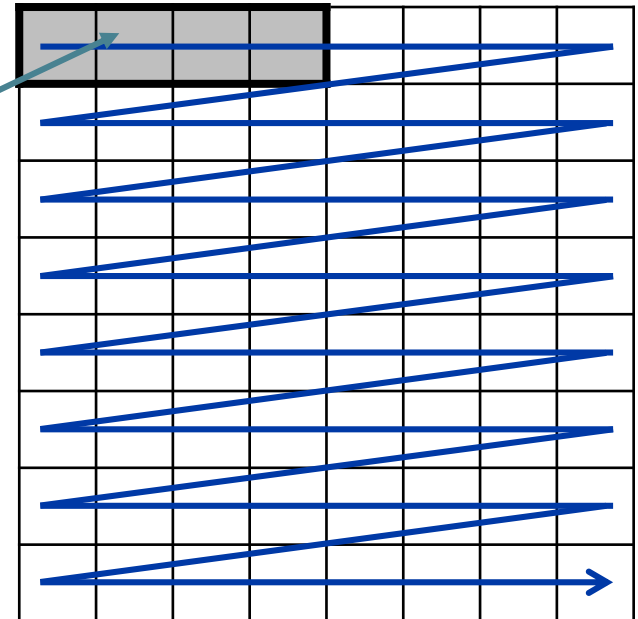


Adding Elements of 2D Array

- Consider the following code that adds the elements of a 2D array

```
float sum = 0.0f;  
for(unsigned int row = 0; row < 4096; ++row) {  
    for(unsigned int col = 0; col < 4096; ++col) {  
        sum += A[row][col];  
    }  
}
```

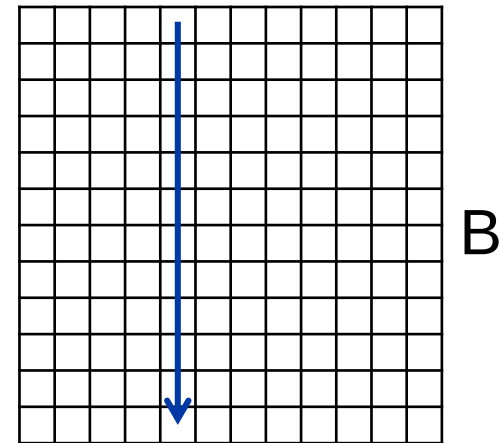
Interchanging the loops
improves spatial locality



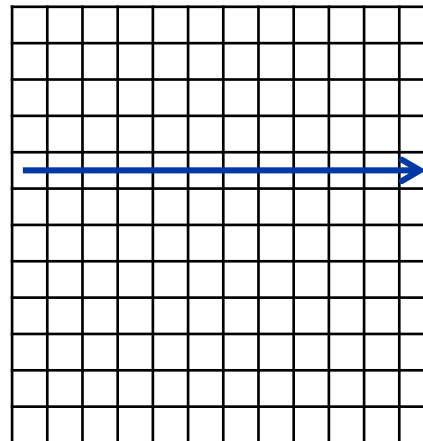
Matrix Multiplication

- Consider the following matrix multiplication code ($C = A \times B$):

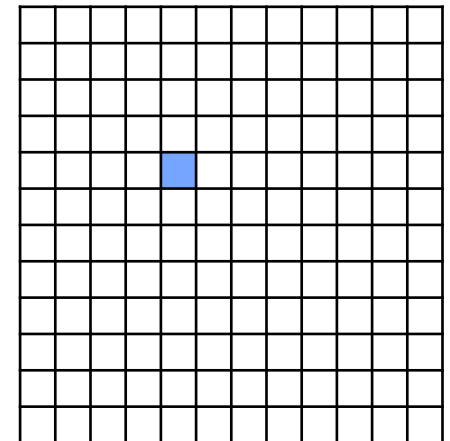
```
for(unsigned int row = 0; row < 2048; ++row) {  
    for(unsigned int col = 0; col < 2048; ++col) {  
        for(unsigned int i = 0; i < 2048; ++i) {  
            C[row][col] += A[row][i]*B[i][col];  
        }  
    }  
}
```



A



C

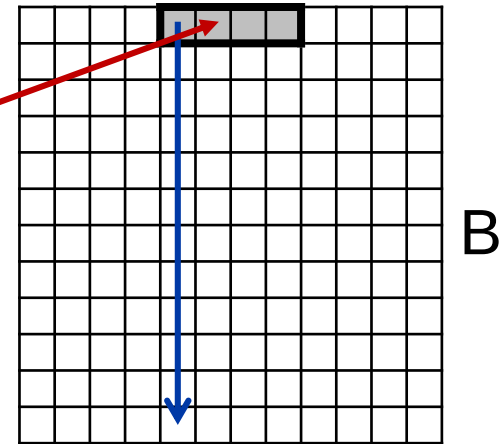


Matrix Multiplication

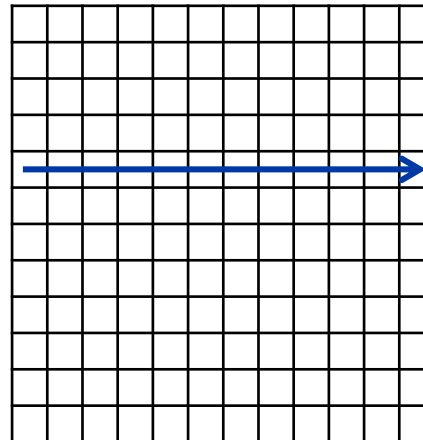
- Consider the following matrix multiplication code ($C = A \times B$):

```
for(unsigned int row = 0; row < 2048; ++row) {  
    for(unsigned int col = 0; col < 2048; ++col) {  
        for(unsigned int i = 0; i < 2048; ++i) {  
            C[row][col] += A[row][i]*B[i][col];  
        }  
    }  
}
```

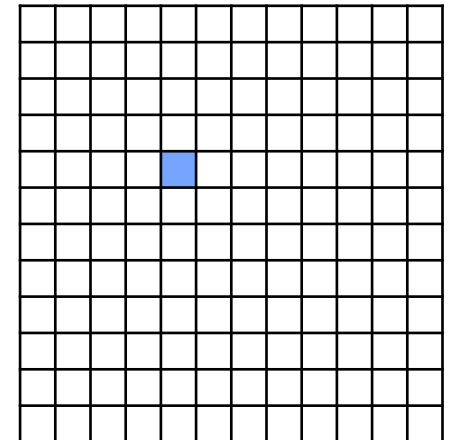
**Data in a previously loaded
cache line will be evicted by the
time we come back for it**



A

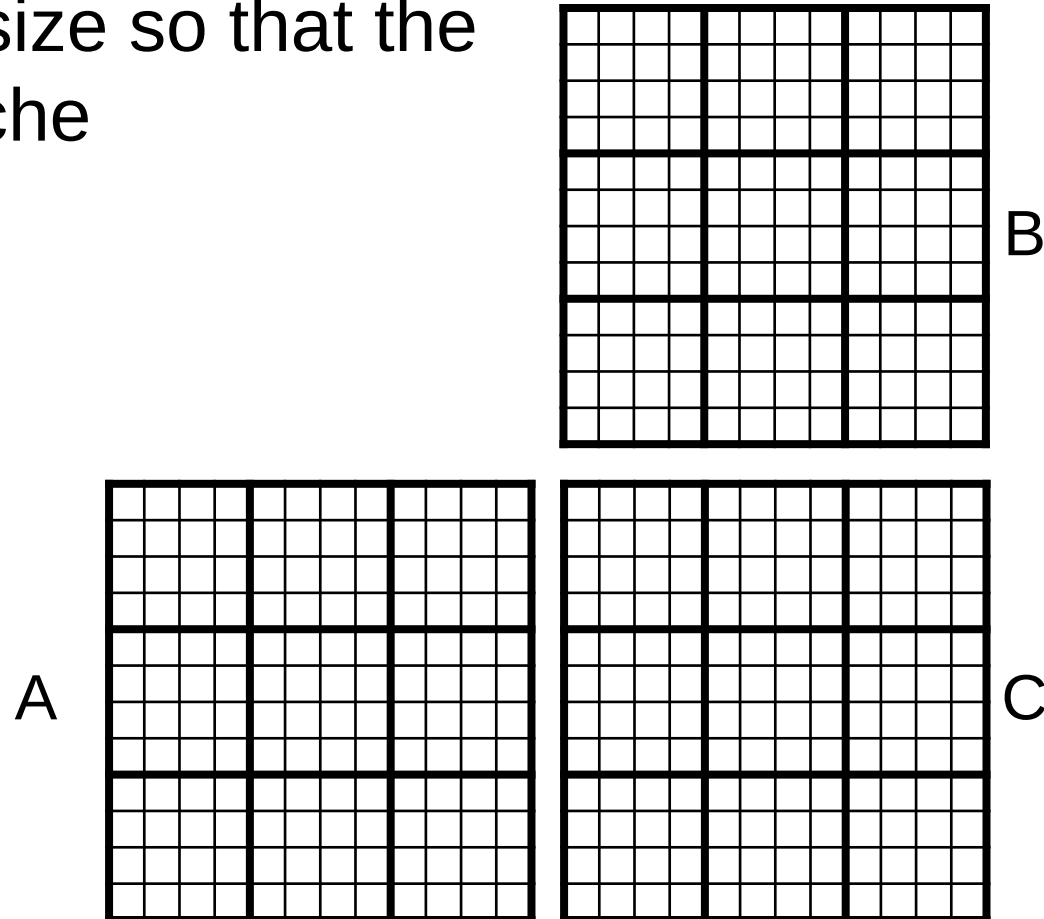


C



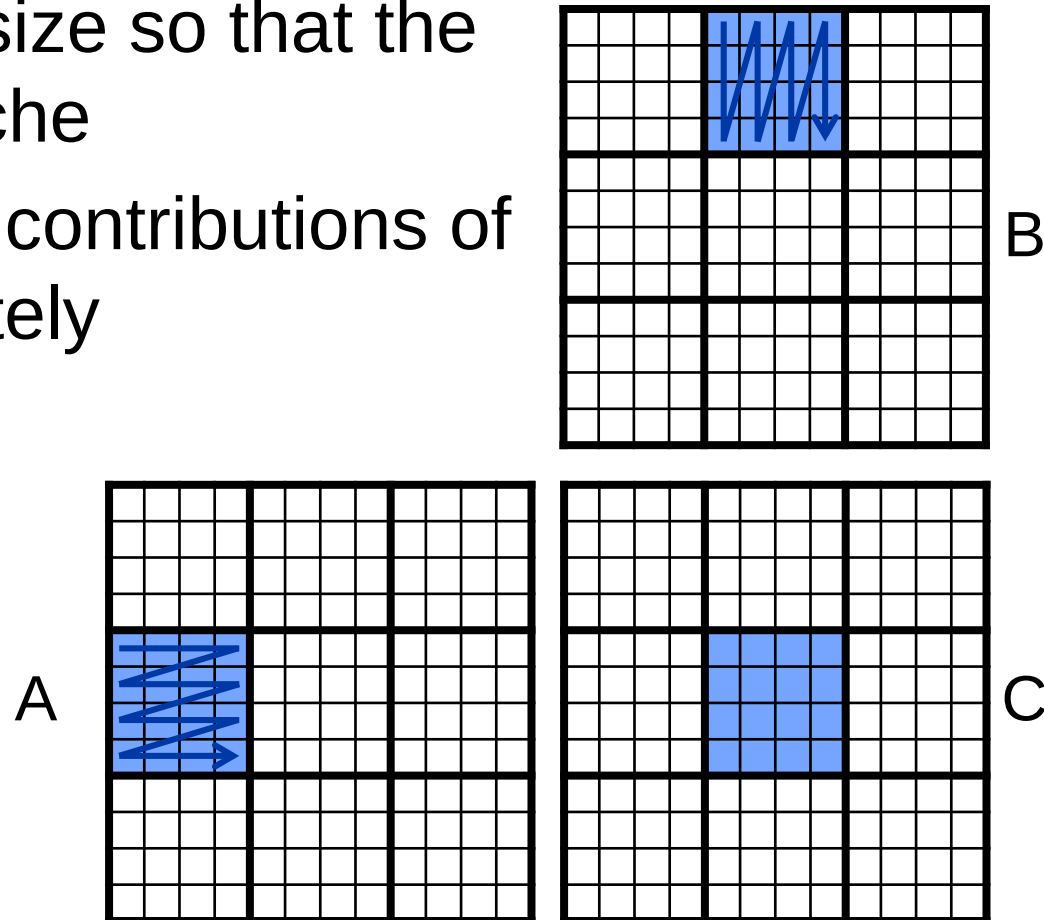
Tiled Matrix Multiplication

- Divide the matrices into tiles
 - Choose the tile size so that the tiles fit in the cache



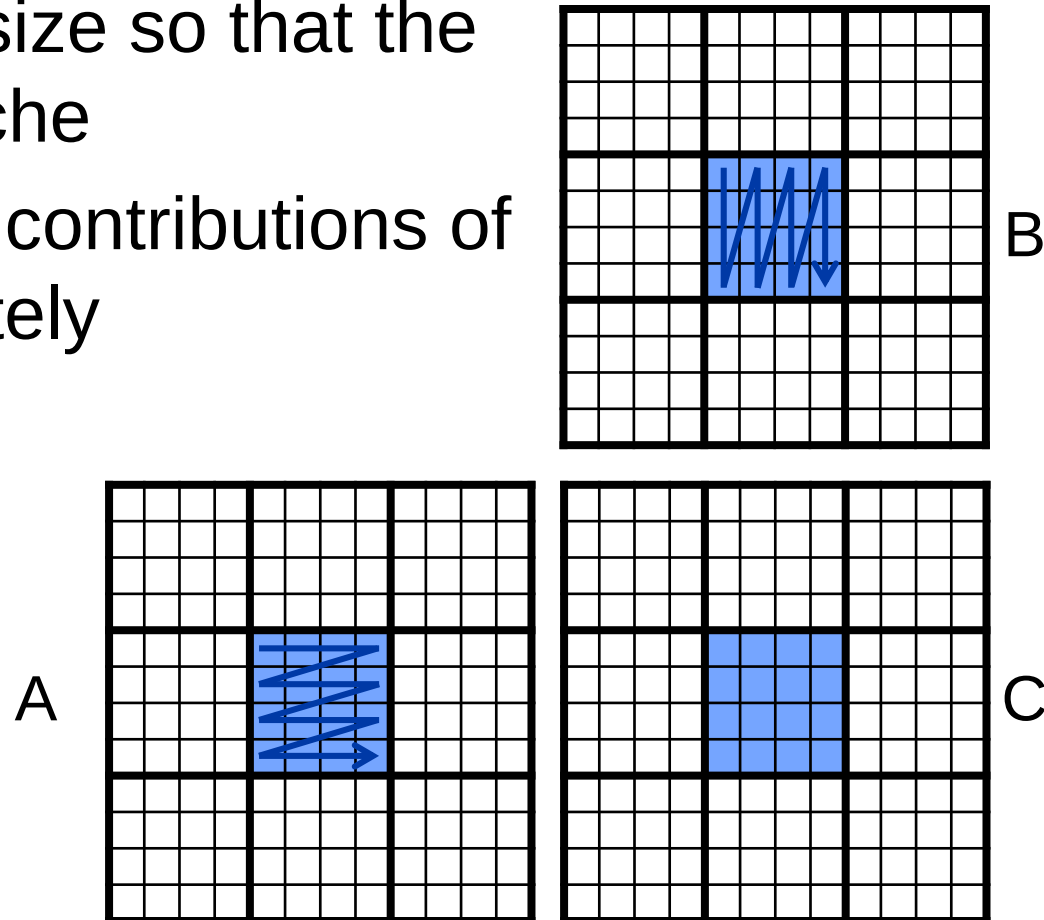
Tiled Matrix Multiplication

- Divide the matrices into tiles
 - Choose the tile size so that the tiles fit in the cache
 - Compute partial contributions of each tile separately



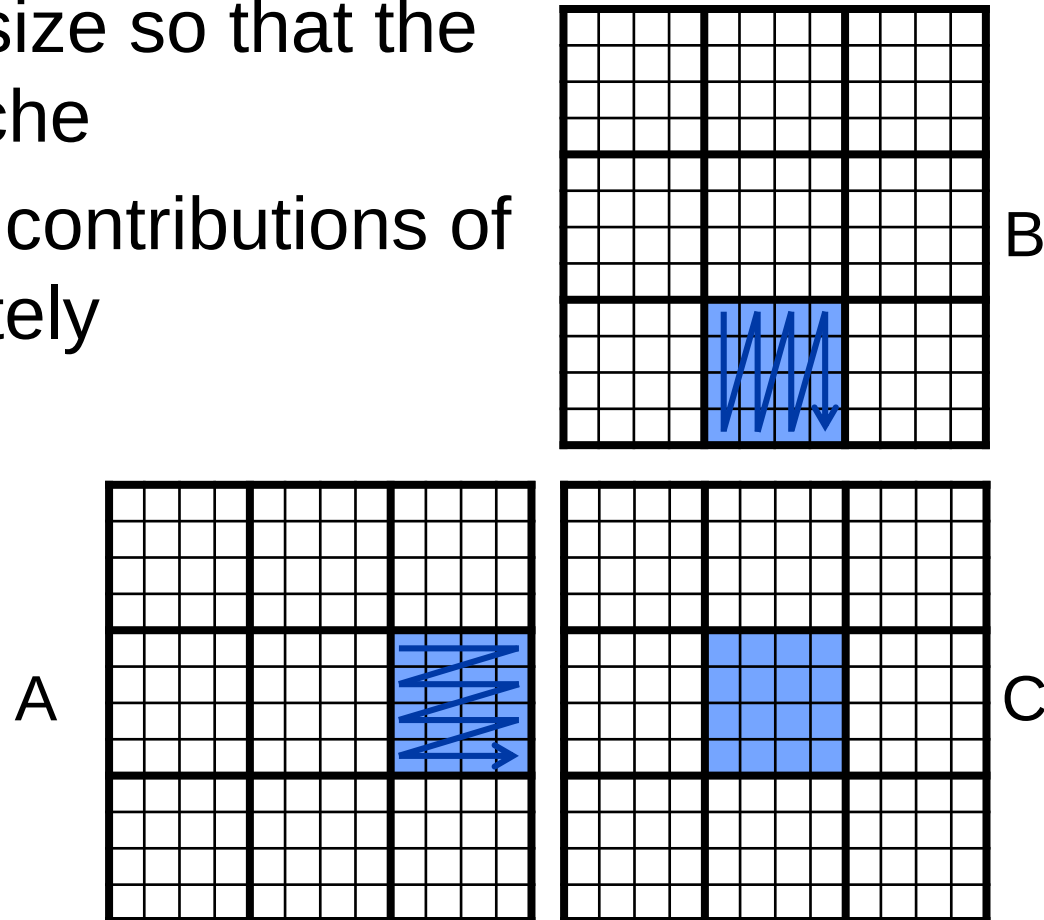
Tiled Matrix Multiplication

- Divide the matrices into tiles
 - Choose the tile size so that the tiles fit in the cache
 - Compute partial contributions of each tile separately



Tiled Matrix Multiplication

- Divide the matrices into tiles
 - Choose the tile size so that the tiles fit in the cache
 - Compute partial contributions of each tile separately



Tiled Matrix Multiplication Code

```
for(unsigned int rowTile = 0; rowTile < 2048/TILE_SIZE; ++rowTile) {
    for(unsigned int colTile = 0; colTile < 2048/TILE_SIZE; ++colTile) {
        for(unsigned int iTile = 0; iTile < 2048/TILE_SIZE; ++iTile) {
            for(unsigned int row = rowTile*TILE_SIZE; row < (rowTile + 1)*TILE_SIZE; ++row) {
                for(unsigned int col = colTile*TILE_SIZE; col < (colTile + 1)*TILE_SIZE; ++col) {
                    for(unsigned int i = iTile*TILE_SIZE; i < (iTile + 1)*TILE_SIZE; ++i) {
                        C[row][col] += A[row][i]*B[i][col];
                    }
                }
            }
        }
    }
}
```

Textbook Sections

- The content in these slides corresponds to:
 - Textbook:
 - *Computer Organization and Design, 5th Edition by David Patterson and John Hennessy, Morgan Kaufmann, 2014.*
 - Sections:
 - 5.10, 5.14